

CPU scheduling*

Chapter 5 in Silberschatz, Galvin &Gagne

Michelle Kuttel

Professor

Department of Computer Science

University of Cape Town



1

The best performance improvement is the transition from the nonworking state to the working state. That's infinite speedup. - John Ousterhout, Stanford University

* Note on pronunciation: UK vs USA

Scheduling is a fundamental function of an operating system

At any point in time, some threads are running on the system's processors, others are waiting their turn, others are waiting for I/O, or on conditions

The **processor scheduling policy** determines which thread runs next.

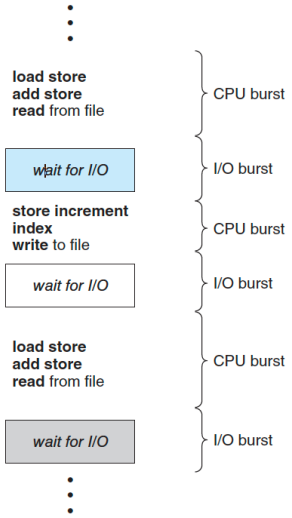


Figure 5.1 Alternating sequence of CPU and I/O bursts.

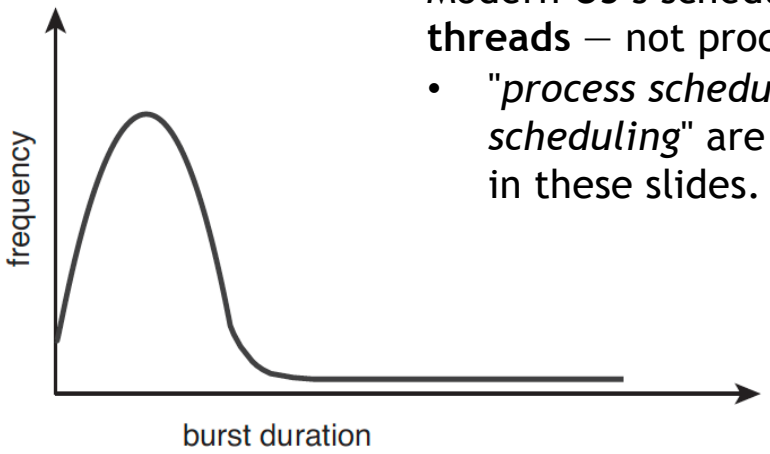


Figure 5.2 Histogram of CPU-burst durations.

Modern OS's schedule **kernel-level threads** — not processes.

- "*process scheduling*" and "*thread scheduling*" are used interchangeably in these slides.



CPU Scheduler

Selects one of the processes in the **ready queue** to be executed.

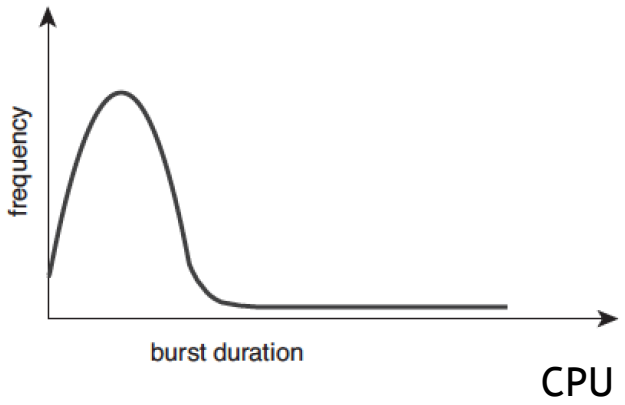
Ready queue can be implemented as:

- FIFO queue
- a priority queue (Windows)
- a tree (Linux uses red-black trees)
- an unordered linked list.

The are ready queue entries represent schedulable tasks/threads.

CPU versus I/O bursts

The durations of CPU bursts vary greatly from process to process and from computer to computer - they tend to have an exponential frequency curve.



Processes are usually either

- **I/O-bound** - many short CPU bursts.
- **CPU-bound** - few long CPU bursts.

This distribution is important when implementing a CPU-scheduling algorithm.

I/O is communication with devices, such as disks, keyboards, networks, and displays.

Input e.g.	Reading from the keyboard, reading a file from disk receiving data over a network
Output e.g.	Printing to the screen writing a file to disk sending data over a network

CPU-bound vs I/O-bound Processes

CPU-bound

Spends most of its time performing computation

Has **long CPU bursts**

Rarely blocks for input/output

Performance limited mainly by **CPU speed**
e.g. scientific simulation, video encoding, image rendering, large numerical calculations

I/O-bound

Spends most of its time waiting for I/O

Has **many short CPU bursts**

Frequently blocks for disk, network, or user input

Performance limited mainly by **I/O speed**
e.g. web server, file transfer, database query, interactive editor or shell

CPU-bound:

```
[======== CPU ========][======== CPU ========][======== CPU ========]
```

I/O-bound:

```
[==CPU==][ I/O wait ][==CPU==][ I/O wait ][==CPU==][ I/O wait ]
```

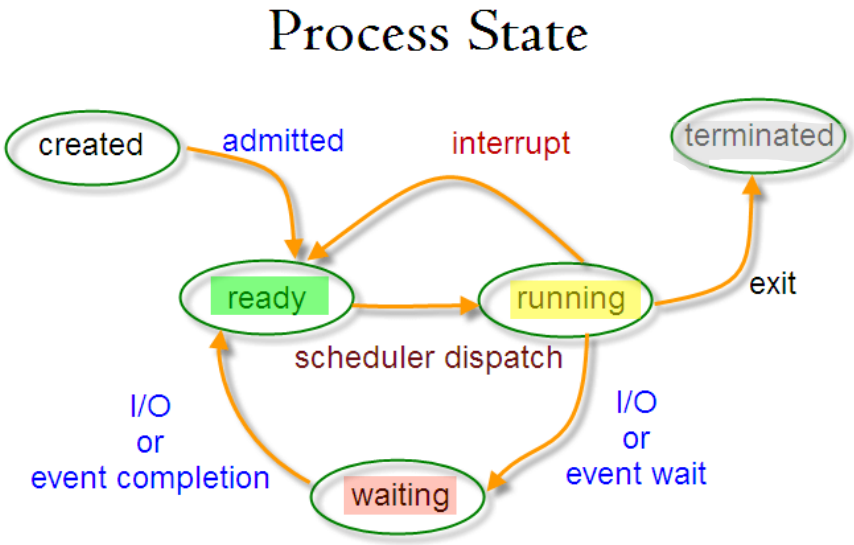
•CPU-bound processes tend to keep the CPU for longer.
•I/O-bound processes often need quick access to the CPU, then block again.

A good scheduler balances **throughput** and **responsiveness** by handling both well: **keeps the CPU busy while other processes are waiting for I/O.**

Pre-emptive and Non-pre-emptive Scheduling

CPU scheduling can take place when a **process switches** from:

1. **running** state to waiting state *e.g I/O start*
2. **running** state to **ready** state *e.g interrupt*
3. waiting state to **ready** state *e.g I/O end*
4. **running** state to **terminated**



Non-pre-emptive/cooperative scheduling if done in **situations 1 and 4 only**.

- Once it has CPU, process keeps it until it is waiting or **terminated**.

Pre-emptive scheduling introduces concurrency hazards, since shared data may be accessed by code that can now be interrupted and resumed.

- But used in all modern OS process scheduling

Context switches: how often?

On UNIX systems, context switches are reported using the **vmstat** command, under the `CS column`

```
vmstat 1 10
```

vmstat [interval] [count]

Can also use the **/proc** file system to determine the number of context switches for a given process.

For example, the contents of the file **/proc/2166/status** will list various statistics for the process with **pid =2166**.

Context switches: how often?

```
$ ps -aef
mkuttel    647     1  0 13:56 ?        00:00:00 /lib/systemd/systemd --user
root       651     2  0 13:56 ?        00:00:00 [kworker/6:1]
mkuttel    652   647  0 13:56 ?        00:00:00 (sd-pam)
mkuttel    698   642  0 13:56 ?        00:00:00 sshd: mkuttel@pts/10
mkuttel    699   698  0 13:56 pts/10   00:00:00 -bash
```

```
$ cat /proc/698/status
…
voluntary_ctxt_switches: 1085
nonvoluntary_ctxt_switches: 5
```

- **voluntary context switch** occurs when a process has given up control of the CPU because it requires a resource that is currently unavailable (such as blocking for I/O.)
- **nonvoluntary context switch** occurs when the CPU has been taken away from a process, such as when its time slice has expired or it has been pre-empted by a higher-priority process.

Scheduling criteria (metrics)

Range of different scheduling algorithms with different properties.

Many potential criteria for comparison, such as:

maximize

- CPU utilization – keep the CPU as busy as possible
- Throughput – number of processes per time unit (e.g., 5 per second)

minimize

- Turnaround time – (= interval from the time of **submission** of a process to the time of **completion**) i.e. total amount of time to execute a particular process
- Waiting time – total amount of time a process has been waiting in the ready queue
- Response time – (= **time to start responding**) amount of time it takes from when a request was submitted until the **first response** is produced, not final output (for time-sharing environment)

Additional Scheduling criteria (metrics)

Other **desirable criteria** are:

Fairness — all runnable processes should get a reasonable share of CPU time

Starvation freedom / bounded waiting — no process should wait indefinitely

Predictability / low jitter — response times should be reasonably consistent

Scheduler overhead — CPU time spent on scheduling and context switching should be low

For interactive systems (such as a laptop), it may be more important to minimize the **variance** in the response time than to minimize the **average** response time.

- A reasonable and predictable response time may more desirable than a faster average, but highly individual response times.

However, little work has been done on CPU-scheduling algorithms that minimize v**ariance**.

Scheduling Algorithms

Concerned with the problem of deciding which of the processes in the **ready queue** is allocated the CPU's core.

Will discuss **5 algorithms**:

We focus on uniprocessor/ **single core** systems.



- First–Come, First-Serve (FCFS)



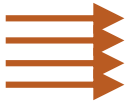
- Shortest-Job-First Scheduling (SJF)



- Round-Robin Scheduling (RR)



- Priority Scheduling

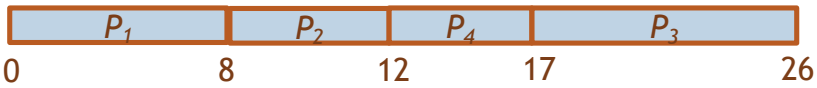


- Multilevel Queue Scheduling

Scheduling algorithm evaluation

Analytical modelling: deterministic evaluation.

- Give exact numbers for comparison
- Simple and fast, but **limited to fixed scenario**



Queueing Models: queueing-network analysis.

- distribution of CPU and I/O bursts measured and then approximated or estimated (probabilistic).
- Knowing arrival rates and service rates, we can compute utilization, average queue length, average wait time, and so on.
  - E.g. Little's formula

- L = average queue length
- W = average waiting time in queue
- λ = average arrival rate into queue
- **Littles Law** – in steady state, processes leaving queue must equal processes arriving, thus:

  $L = \lambda \times W$

  - Valid for any scheduling algorithm and arrival distribution

    e.g. customers arrive at **2 per minute**
    each customer spends **5 minutes** in the system

    $L = 2 \times 5 = 10$

    So on average, there are
    10 customers in the system.

Simulations: program a model of the computer system.

- use random-number generator to generate processes, CPU burst times, arrivals, departures, and so on, according to probability distributions.
- As simulation executes, generates statistics of algorithm performance.

Actual implementation

First-Come, First-Served (FCFS) Scheduling



Managed with FIFO queue.

Arriving process PCB added to end of the queue;

Process at head of queue scheduled next on CPU.

Advantages:

- simple
- **Fair**

Disadvantages: average Waiting time /average **response time** can be **long.**

In some situations, the worst possible response time.

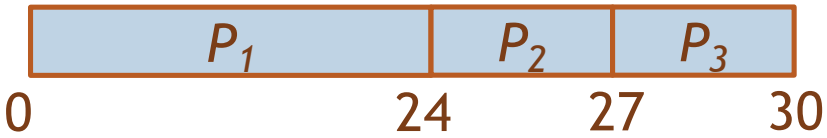
First-Come, First-Served (FCFS) Scheduling



Consider:

Process	Burst time
P_1	24
P_2	3
P_3	3

Gantt Chart



If processes arrive in the order:
P_1 , P_2 , P_3

P_2 24

P_3 27

Note my variation on the scheduling visualization to avoid common errors with waiting time

Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
Average Waiting time : (0 + 24 + 27)/3 = 17

First-Come, First-Served (FCFS) Scheduling



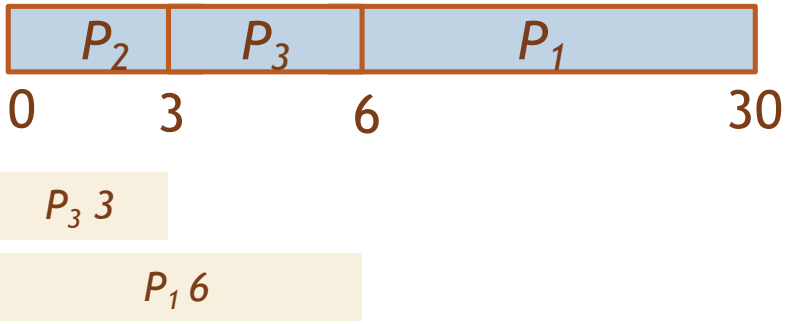
Consider:

Process	Burst time
P_1	24
P_2	3
P_3	3

If processes arrive in the order:

P_2 , P_3 , P_1

Gantt Chart



Waiting time for P_2 = 0; P_3 = 3, P_1 = 6;

Average Waiting time : (0 + 3 + 6)/3 = 3

First-Come, First-Served (FCFS) Scheduling



Managed with FIFO queue.
Arriving process PCB added to end of the queue;
Process at head of queue scheduled next on CPU.

Advantage: **simple** to understand and implement
Disadvantages: average Waiting time can be **long.**

FCFS is non-pre-emptive

Also Convoy effect - short process behind long process

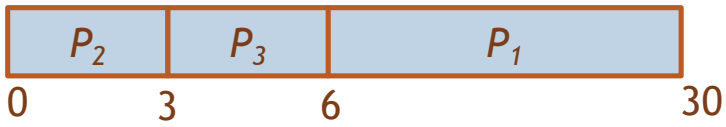
- Consider one CPU-bound and many I/O-bound processes

Consider:

Process	Burst time
P_1	24
P_2	3
P_3	3

If instead processes arrive in the order:
P_2 , P_3 , P_1

Gantt Chart



Waiting time for P_2 = 0; P_3 = 3, P_1 = 6;
Average Waiting time : (0 + 3 + 6)/3 =3

First-Come, First-Served (FCFS) Scheduling



FCFS is Non-pre-emptive

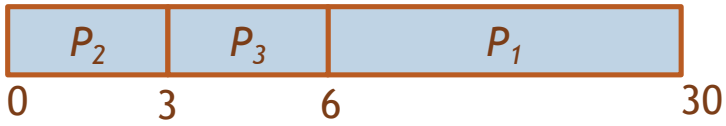
- Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/O.
- This is very **bad for interactive systems**, where it is important that each process get a **share of the CPU** at regular intervals. It would be disastrous to allow one process to keep the CPU for an extended period

Consider:

Process	Burst time
P_1	24
P_2	3
P_3	3

If instead processes arrive in the order:
P_2 , P_3 , P_1

Gantt Chart



Waiting time for P_2 = 0; P_3 = 3, P_1 = 6;
Average Waiting time : (0 + 3 + 6)/3 =3

Shortest-Job-First Scheduling (SJF)



Actually *shortest-next-CPU-burst* algorithm

Associate with each process the length of its next CPU burst

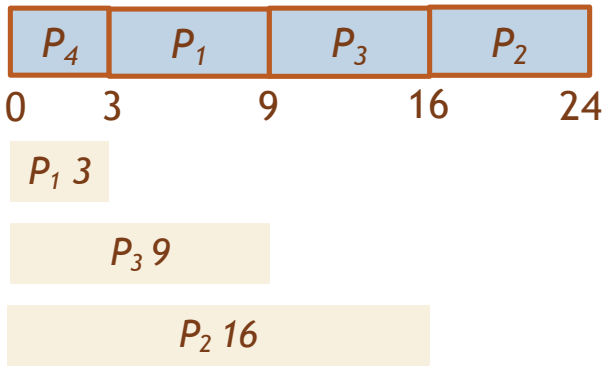
- Use these lengths to schedule the process with the shortest time

SJF is provably **optimal for waiting time:**
minimum average Waiting time for a given set of processes

Consider:

Process	Burst time
P_1	6
P_2	8
P_3	7
P_4	3

Gantt Chart



Average Waiting time : (0 + 3 + 9 + 16)/4 = 7
Compare FCFS: 10.25 ms

Shortest-Job-First Scheduling (SJF)



Accurate implementation is **impossible.**

Approximate implementation is **difficult:**

- hard to predict the length of the next CPU burst.

Assume that the next CPU burst will be similar to previous

- Predict length of next CPU burst as exponential average of previous.
- pick the process with the shortest predicted CPU burst.

1. t_n = actual length of nth CPU burst
2. τ_{n+1} = predicted value for the next CPU burst
3. $\alpha, 0 \le \alpha \le 1$
4. Define :

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n.$$

α is weighting, usually set to ½

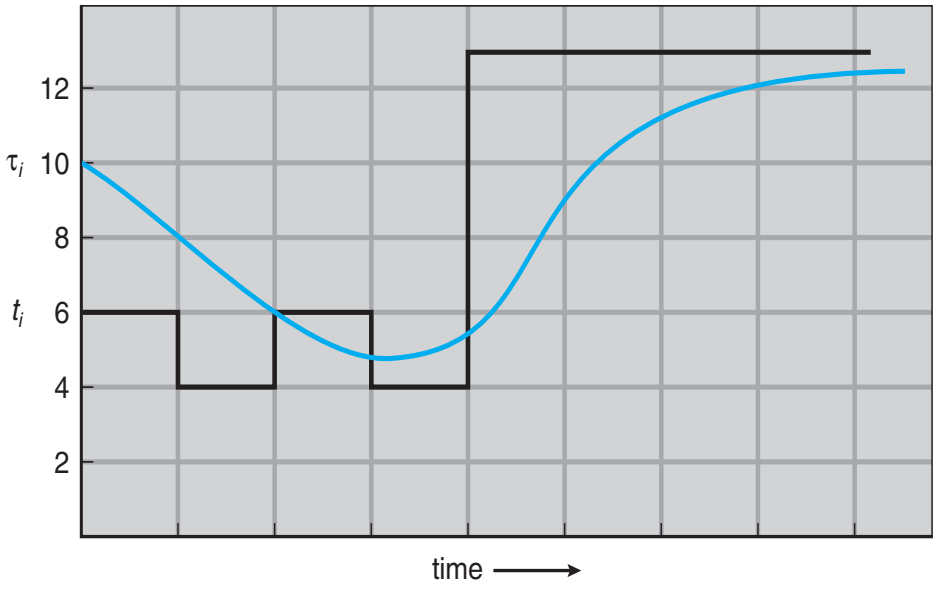
history

initial τ_0 can be defined as a constant or an overall system average.

Shortest-Job-First Scheduling (SJF)

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n.$$

α set to ½



CPU burst (t_i)		6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

If we expand the formula, we get:

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\alpha\, t_{n-1} + \ldots$$
$$+ (1-\alpha)^j \alpha\, t_{n-j} + \ldots$$
$$+ (1-\alpha)^{n+1} \tau_0$$

- Since both α and $(1-\alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

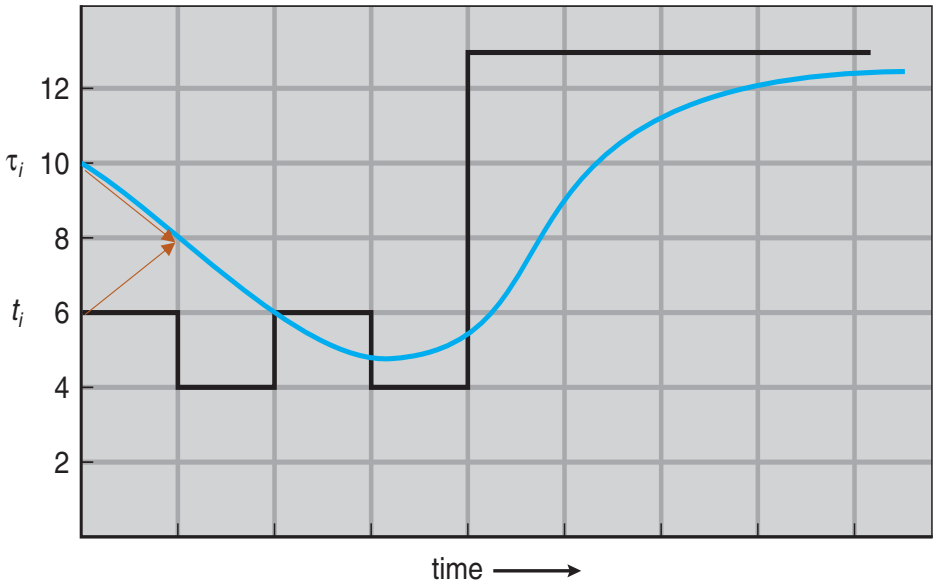
If $\alpha = 0$, $\tau_{n+1} = \tau_n$
- Most recent burst does not count

If $\alpha = 1$, $\tau_{n+1} = t_n$
- Only the actual last CPU burst counts

Shortest-Job-First Scheduling (SJF)

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\tau_n.$$

α set to ½



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10 → 8	6	6	5	9	11	12	...

0.5*10+0.5*6=8

If we expand the formula, we get:

$$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\alpha\, t_{n-1} + \ldots$$
$$+ (1-\alpha)^j \alpha\, t_{n-j} + \ldots$$
$$+ (1-\alpha)^{n+1} \tau_0$$

- Since both α and $(1-\alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

If $\alpha = 0$, $\tau_{n+1} = \tau_n$
- Most recent burst does not count

If $\alpha = 1$, $\tau_{n+1} = t_n$
- Only the actual last CPU burst counts

Pre-emptive Shortest-Job-First Scheduling (SJF)

SJF is either **pre-emptive** or **non-pre-emptive.**

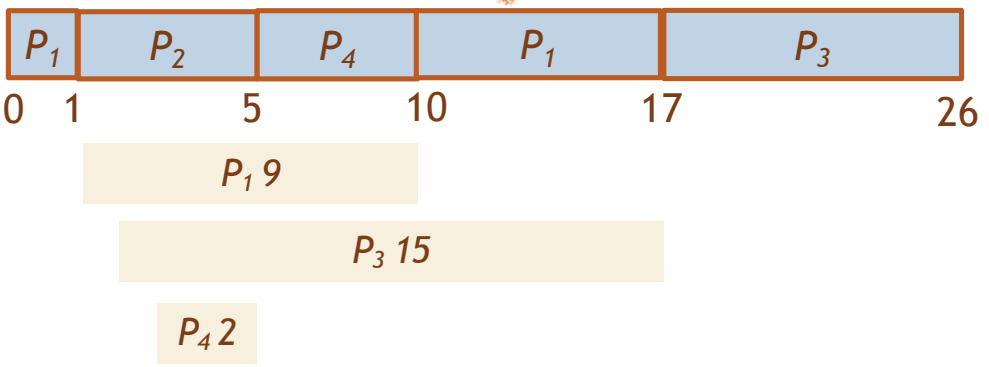
shortest-remaining-time-first

If the **predicted CPU burst** of a newly arrived process is **shorter** than what is left of the currently executing process, a pre-emptive SJF algorithm will pre-empt the currently executing process, whereas a non-pre-emptive algorithm will allow the currently running process to finish its CPU burst.

Consider:

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Gantt Chart



Average Waiting time : ((10-1)+(1-1)+(17-2)+(5-3))/4 = 6.5 ms

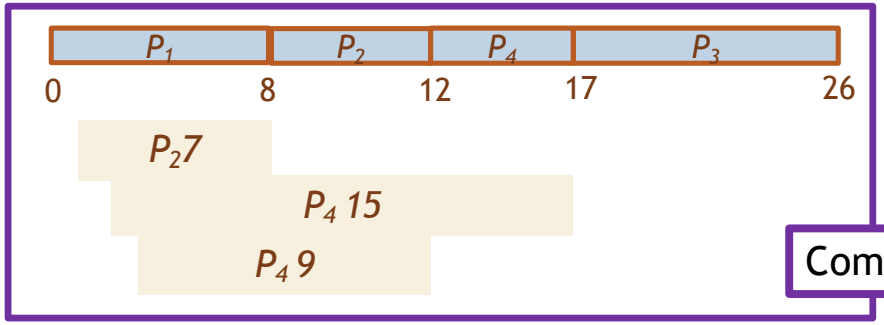
Pre-emptive Shortest-Job-First Scheduling (SJF)



SJF is either **pre-emptive** or **non-pre-emptive.**

shortest-remaining-time-first

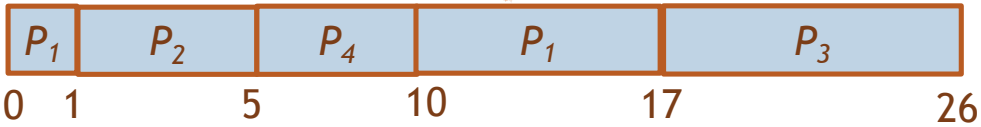
If the predicted CPU burst of a newly arrived process is shorter than what is left of the currently executing process, a pre-emptive SJF algorithm will pre-empt the currently executing process, whereas a non-pre-emptive algorithm will allow the currently running process to finish its CPU burst.



Consider:

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Gantt Chart



Average Waiting time : ((10-1)+(1-1)+(17-2)+(5-3))/4 = 6.5 ms

Compare non-pre-emptive SJF: 7.75 ms

Pre-emptive Shortest-Job-First Scheduling (SJF)



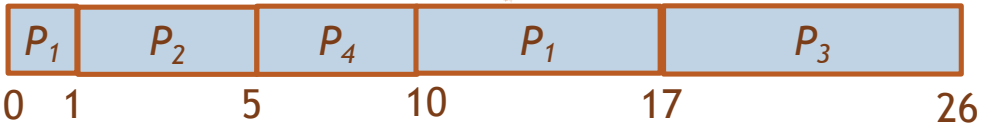
SJF is either **pre-emptive** or non-pre-emptive.

shortest-remaining-time-first

Consider:

Process	Arrival time	Burst time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

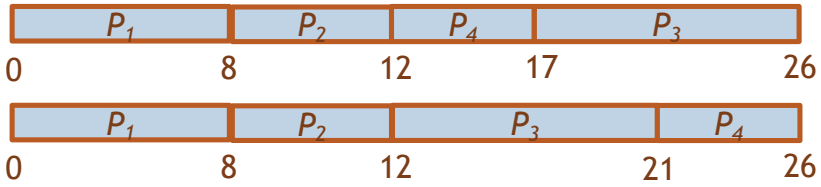
Gantt Chart



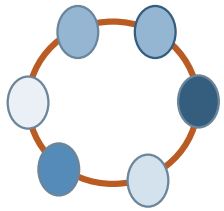
Average Waiting time : ((10-1)+(1-1)+(17-2)+(5-3))/4 = 6.5 ms

Compare non-pre-emptive SJF: 7.75 ms

Compare **FCFS**: 8.75 ms



Round-Robin scheduling (RR)



Similar to FCFS scheduling, but with added pre-emption.

Each process gets **a small slice of CPU time**

(time quantum, q).

q *is* generally 10 to 100 ms

After this, process is **pre-empted** and added to end of the **ready queue**.

- ready queue treated as a circular queue
- new processes are added to 'end' of queue

Timer interrupts every quantum to schedule next process

Performance **depends heavily on size of quantum**

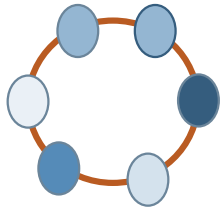
- q large $\Rightarrow$ FIFO
- q small $\Rightarrow$ q must be large with respect to context switch, otherwise overhead is too high (too many switches)

For N processes in the ready queue and time quantum q:

- each process gets $1/N$ of the CPU time in chunks of at most q time units at once.

If a CPU burst is less than 1 time quantum, the process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue.

Round-Robin scheduling (RR)

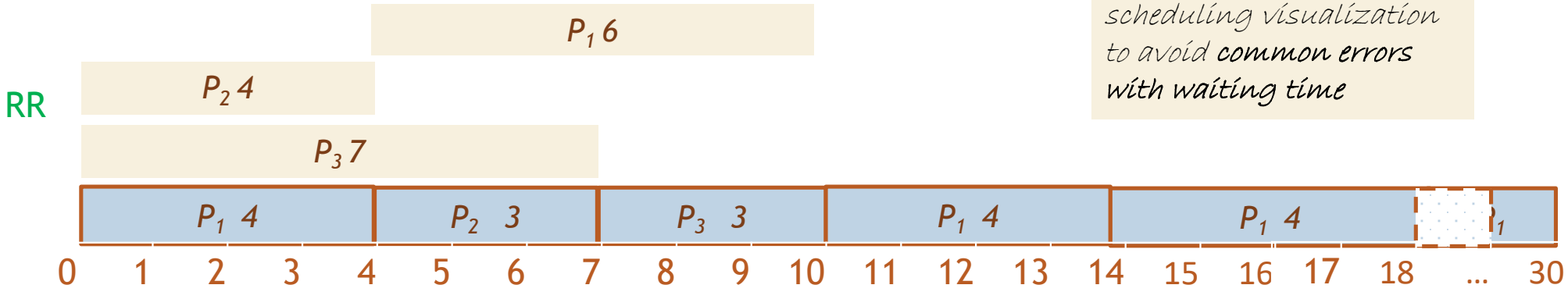


Consider:

time quantum $q = 4$

Process	Burst time
P_1	24
P_2	3
P_3	3

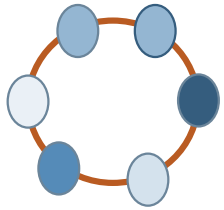
Note my variation on the scheduling visualization to avoid common errors with waiting time



Average Waiting time : (6+4+7)/3 = 17/3 = 5.66 ms
Compare non-pre-emptive SJF: (0+3+6)/3 = 3 ms
Compare FCFS: (0+24+27)/3 = 17 ms (**convoy effect**)
The average Waiting time under the RR policy is **often long.**
Also higher average Turnaround time than SJF, but (usually) **better** Response time.



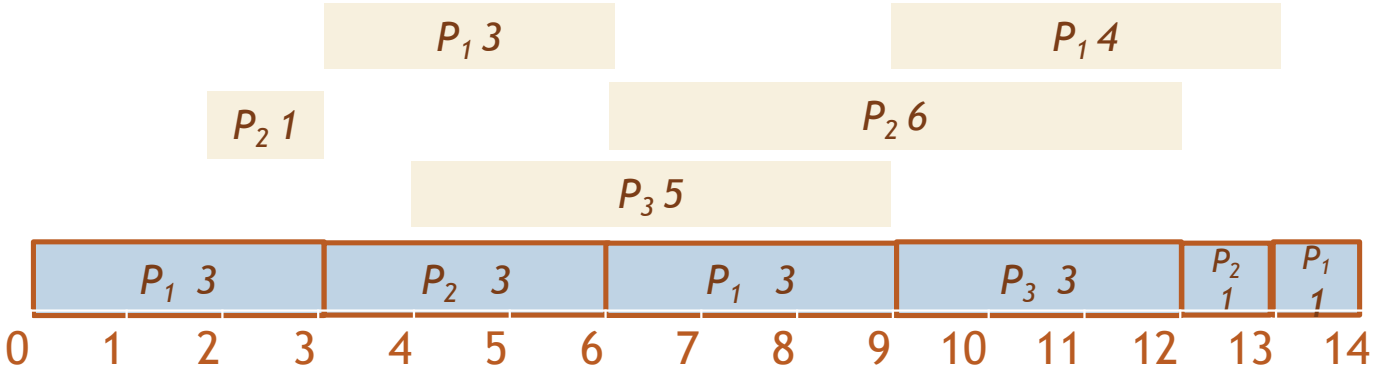
RR – another example (corrected)



time quantum $q = 3$

Process	Arrival Time	CPU Burst time
P1	0	7
P2	2	4
P3	4	3

RR

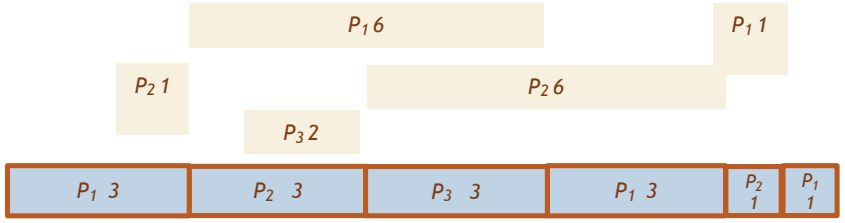


RR Average Waiting time : (7+7+5)/3 =19/3=6.33



Compare pre-emptive SJF

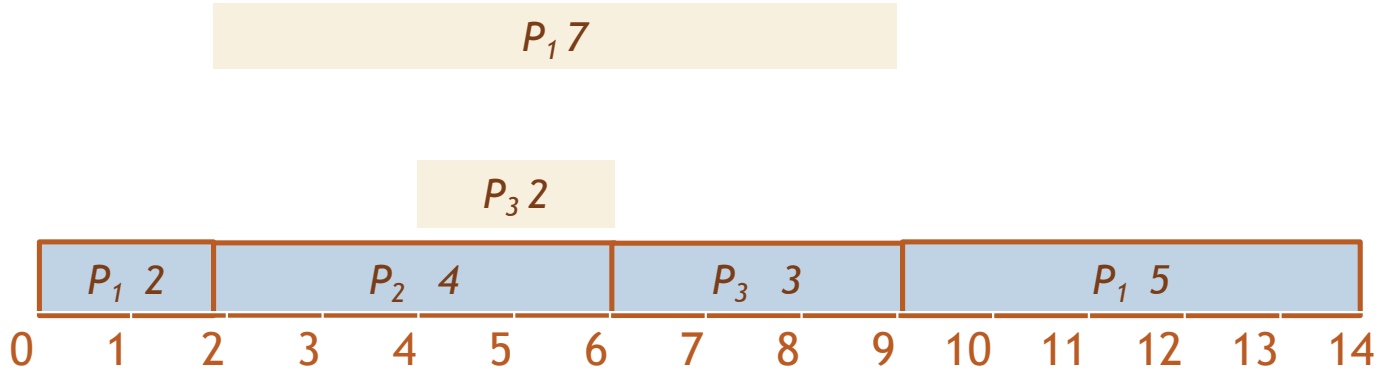
RR



RR Average Waiting time : = 5.33 ms

Process	Arrival Time	CPU Burst time
P1	0	7
P2	2	4
P3	4	3

SJF pre-emptive

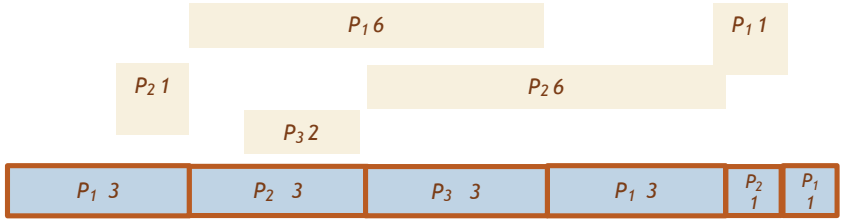


Average Waiting time SJF: (7+2)/3 = 9/3 = 3 ms



Compare FCFS

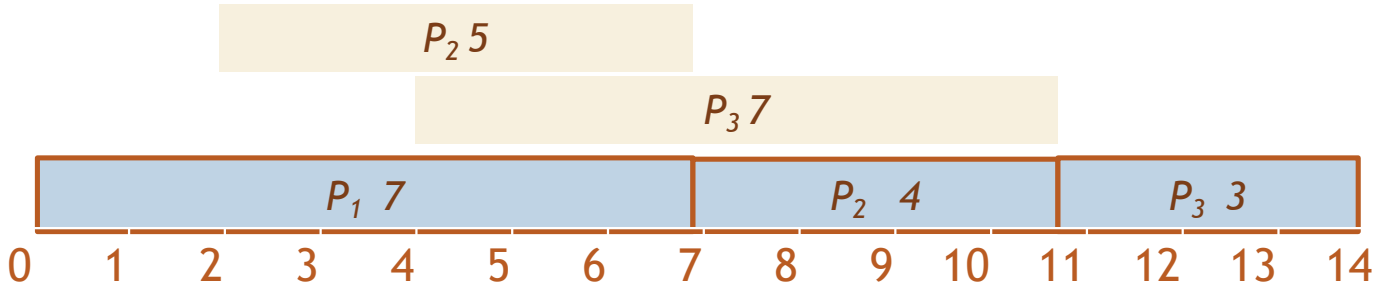
RR



RR Average Waiting time : = 5.33 ms

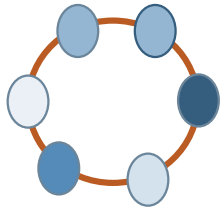
Process	Arrival Time	CPU Burst time
P1	0	7
P2	2	4
P3	4	3

FCFS



Average Waiting time FCFS: (5+7)/3 = 12/3 = 4 ms

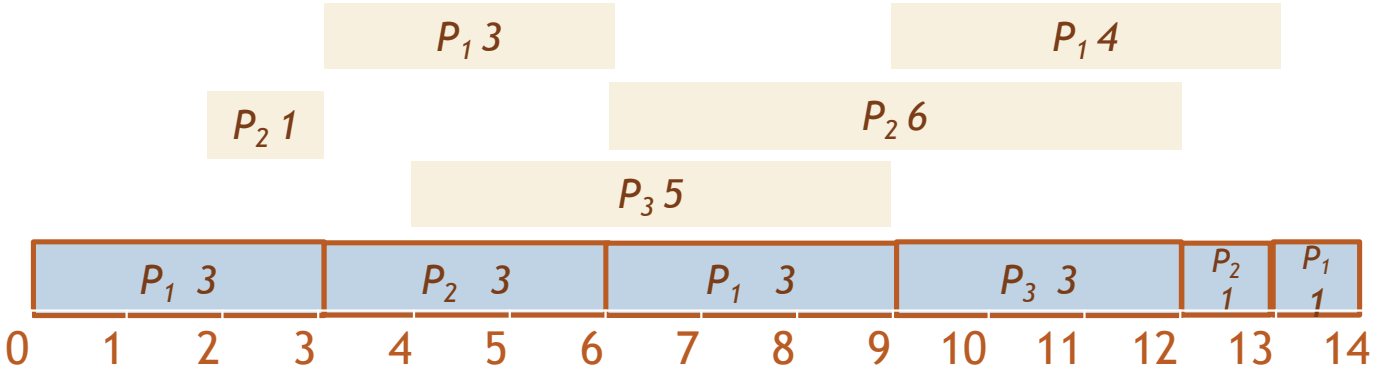
RR – turnaround time (corrected v3!)



time quantum $q = 3$

Process	Arrival Time	CPU Burst time
P1	0	7
P2	2	4
P3	4	3

RR



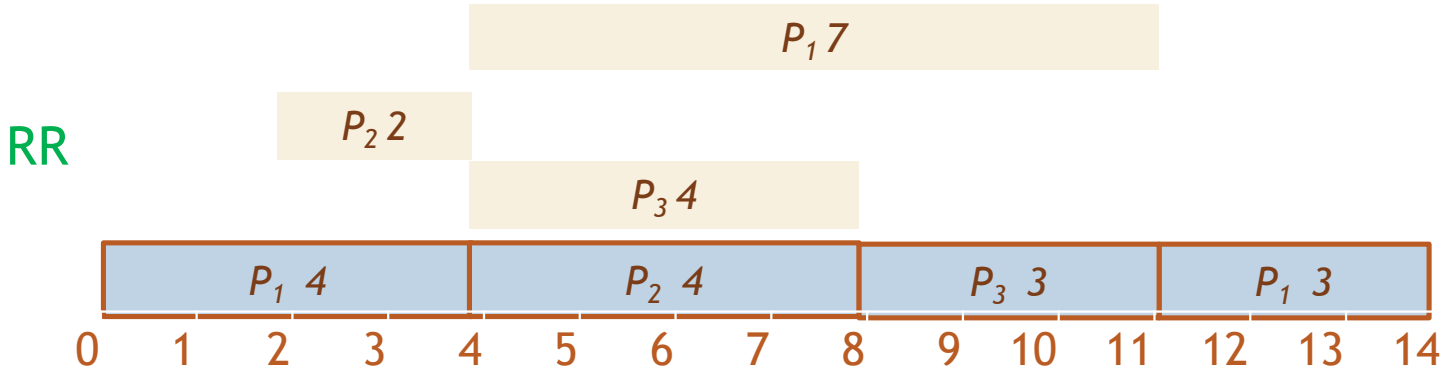
RR Average Turnaround time : (14+(13-2)+(9-4))/3 =30/3=10



RR – turnaround time

time quantum ***q = 4***

Process	Arrival Time	CPU Burst time
P1	0	7
P2	2	4
P3	4	3

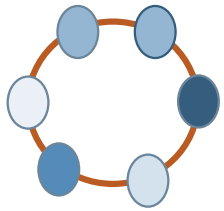


RR Average Turnaround time : (14+(8-2)+(11-4))/3 = 9

RR Average Waiting time : (7+2+4)/3 = 4.33

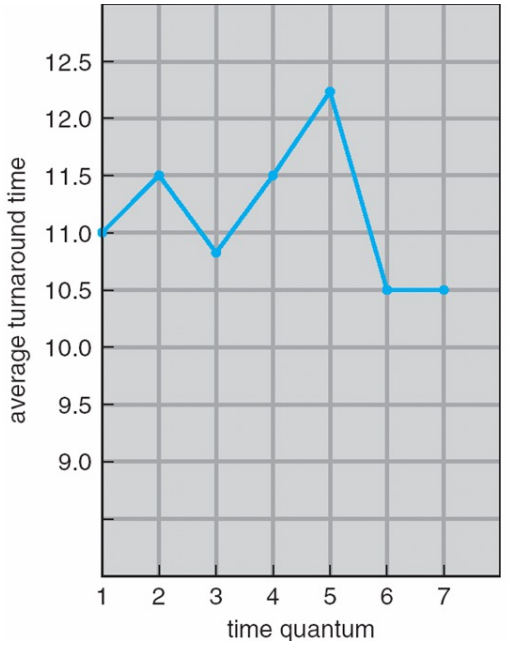
RR Average q=3 Waiting time : ((6+1)+(1+6)+(2))/3 = 16/3= 5.33 ms

Round-Robin scheduling (RR)



Average Turnaround time of a set of processes does **not necessarily improve** as the size of the time-quantum increases.

The average turnaround time can usually be improved if most processes finish their next CPU burst in a single time quantum.



process	time
P_1	6
P_2	3
P_3	1
P_4	7

A rule of thumb is that 80 percent of the CPU bursts should be shorter than the time quantum.

But it depends….

Priority Scheduling



A priority number (integer) is associated with each process.

CPU allocated to the process with the highest priority

- **Pre-emptive**
- Non-pre-emptive

usually smallest ≡ highest priority)

SJF is actually priority scheduling, where priority is the inverse of predicted next CPU burst time.

Priorities can be assigned

- **internally** (e.g. estimated CPU burst, memory requirements, I/O behaviour, or job type) or
- **externally** (importance of process etc.).

Problem: Starvation **(indefinite blocking)**- a low priority process may never execute.

Solution: Aging - as time progresses increase the priority of the process

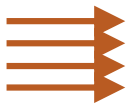
Can combine priority scheduling and RR:

- System executes the highest priority process; processes with the same priority will be run using RR.

starvation

Rumour has it that when they shut down the IBM 7094 at MIT in 1973, they found a low-priority process that had been submitted in 1967 and had not yet been run!

Multilevel Queue Scheduling



For priority scheduling, O(n) search may be necessary to determine the highest-priority process.

easier to have separate queues for each distinct priority, priority scheduling schedules the process in the highest-priority queue.

A process is then **permanently in a given queue.**

Can also partition according to **type of process**

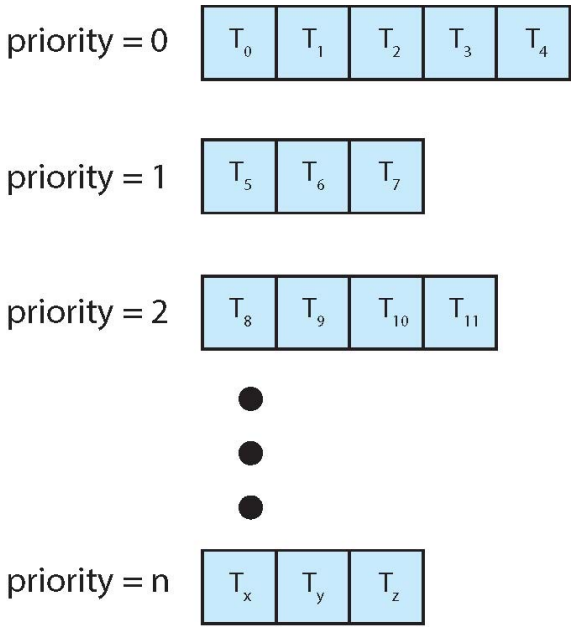
eg. background /foreground

Each queue has its own scheduling algorithm:

- foreground – RR
- background – FCFS

Scheduling must be done between the queues:

- Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
- Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes;
  - i.e., 80% to foreground in RR
  - 20% to background in FCFS



Multilevel feedback queue

This approach allows a process to move between queues.

- Processes are assigned a queue according to the characteristics of their CPU bursts
  - Higher priority queues have I/O-bound and interactive processes (short CPU bursts)
- **aging** can be implemented this way

Multilevel-feedback-queue scheduler defined by the following parameters:

- number of queues
- scheduling algorithms for each queue
- method used to determine when to upgrade a process: a process that waits too long in a lower-priority queue may be moved to a higher-priority queue
- method used to determine when to demote a process: If a process uses too much CPU time, it will be moved to a lower-priority queue.
- method used to determine which queue a process will enter when that process needs service

Done in Windows Operating System

Multilevel queue: process stays in one queue permanently.
Multilevel **feedback** queue: process can move between queues, based on behaviour.

Scheduling algorithms

Algorithm	Pre-emptive?	Strength	Weakness
FCFS	No	Simple, fair	Poor response time
SJF	Either	Best average waiting time	Needs burst prediction
RR	Yes	Good responsiveness	More overhead
Priority	Either	Expresses importance	Starvation
Multilevel queue	Usually mixed	Supports workload classes	Starvation

From classical scheduling models to real schedulers

The previous algorithms are useful **conceptual models**

In practice, operating systems combine several ideas

- Linux CFS is a modern practical scheduler rather than a direct implementation of FCFS, SJF, or RR

CFS: Completely Fair process Scheduling in Linux *(covered in Section 5.7.1)*

CFS based on **scheduling classes:** each **class** has a **priority**

Standard Linux kernels have 2 classes:

(1) a **default scheduling class**

(2) a **real-time scheduling class** with higher priority

Scheduler picks **highest-priority task** in **highest-priority** scheduling **class** to run next

The "standard" Linux kernel has this CPU scheduling algorithm.
However, anyone is free to modify or replace the scheduler to support a different policy.
[This may explain why last year I said that you could not rely on assigned thread priorities?]

CFS: *nice* values

Tasks are given a **proportion of CPU processing time**, not a fixed **time quantum**, called a **targeted latency**

calculated based on the ***nice value*** assigned to each task.

default 0, range −20 to +19,

lower is higher relative priority.

nice processes finish last!

The **targeted latency** is a time interval in which every runnable task should run at least once.

- Proportions of CPU time are allocated from this value
- Value can increase if system is overloaded

latency = how long you have to wait

CFS: priorities

The CFS scheduler CFS uses **virtual runtime**, influenced by the task's **nice value**,to assign priorities;

it records **virtual run time** of each task has run in variable `vruntime`.

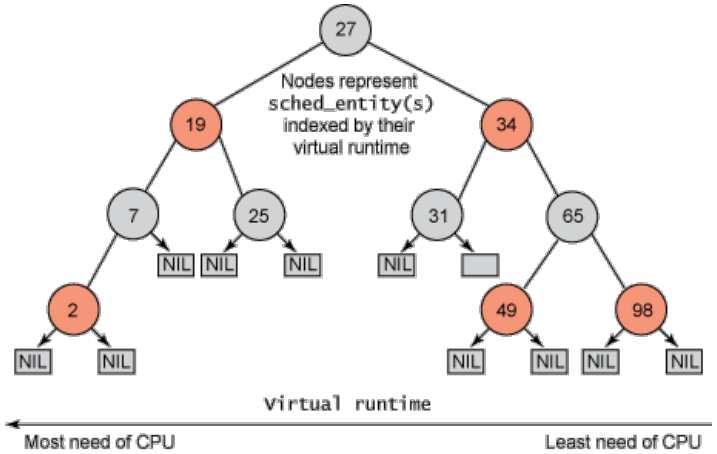
CFS scales a task's accumulated runtime according to its weight (derived from its nice value), so higher-priority tasks accumulate virtual runtime more slowly.

for 0 nice `vruntime` = actual run time.

lower-priority tasks -> `vruntime` larger

higher-priority tasks -> `vruntime` smaller

scheduler selects the task with smallest `vruntime` to run next



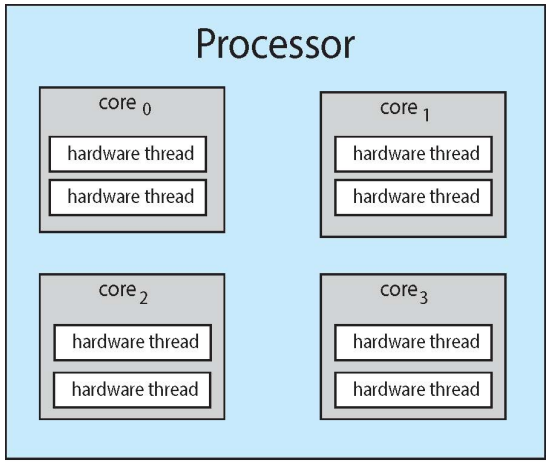
https://developer.ibm.com/tutorials/l-completely-fair-scheduler/

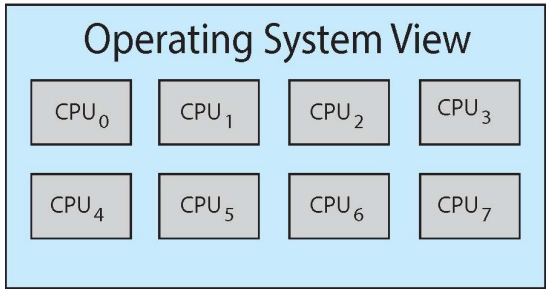
Each runnable task is placed in a **red-black tree**—a balanced binary search tree whose key is based on the value of `vruntime` .
The scheduler picks the **left-most node** of the red-black tree.

*Reminder: A **red-black tree** is a self-balancing binary search tree.*
It stores items in sorted order like an ordinary binary search tree, but it keeps itself roughly balanced so that searching, inserting, and deleting stay efficient.

Multiprocessor Scheduling

Multicore processors have **multiple computing cores** on the **same physical chip.**





Multiple cores/CPUS

means more **complex scheduling**

No single best solution - it ***depends***

Multicore metrics

- **Load balance** — on multicore systems, work should be spread evenly across cores
- **Processor affinity** — keeping a thread on the same core can improve cache performance

Multiprocessor CPU Scheduling

We assume homogeneous processors: all processors identical

Two possible approaches:

Asymmetric multiprocessing

All scheduling decisions, I/O, and other system activities are **handled** by a **single processor.**

The other processors **only execute user code**.

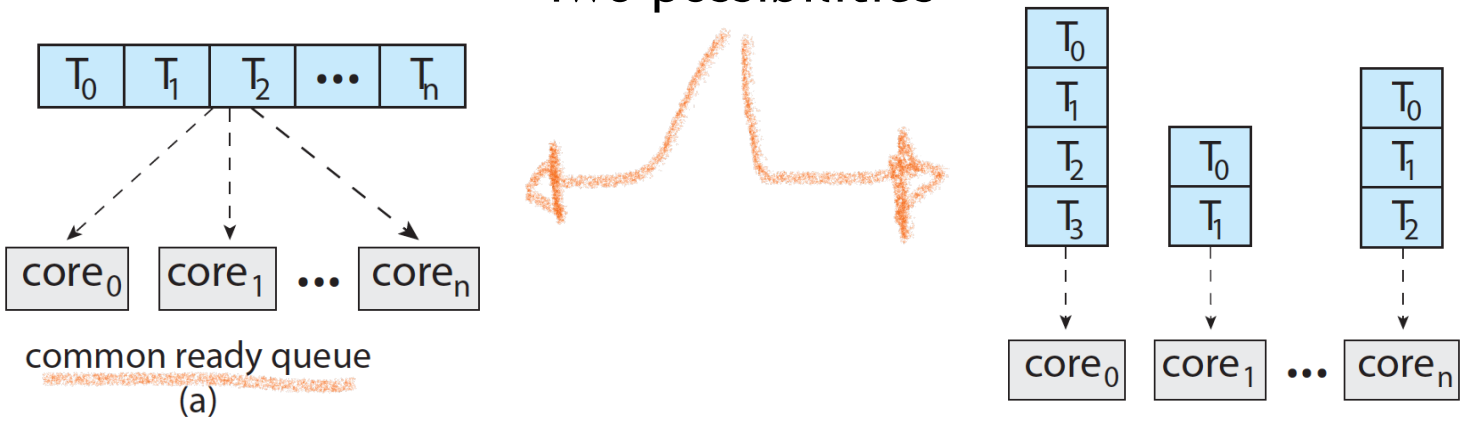
Symmetric multiprocessing (SMP)

Each processor is **self-scheduling.**

Virtually all modern operating systems support SMP, including Windows, Linux, and macOS as well as mobile systems including Android and iOS.

SMP scheduling

Two possibilities



Simpler global choice, but **lock contention**

Race condition on the shared ready queue, must ensure that:

- two separate processors do not choose to schedule the same thread and
- threads are not lost from the queue.

locks are **highly contended**

(synchronisation in the next section)

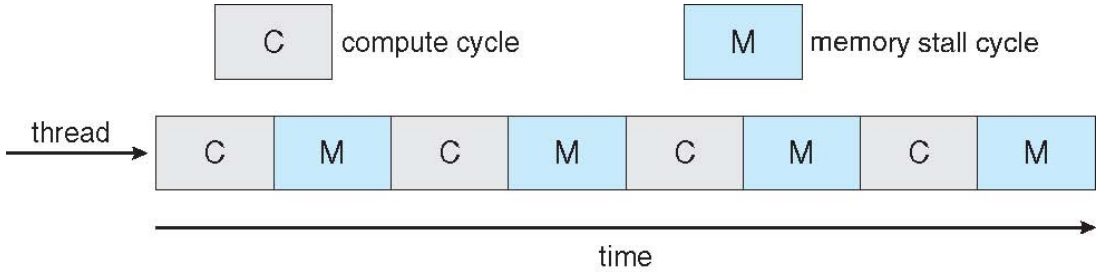
Most common approach on SMPs.

- better scalability/cache locality
- needs load balancing

Memory Stall in Multicore Systems

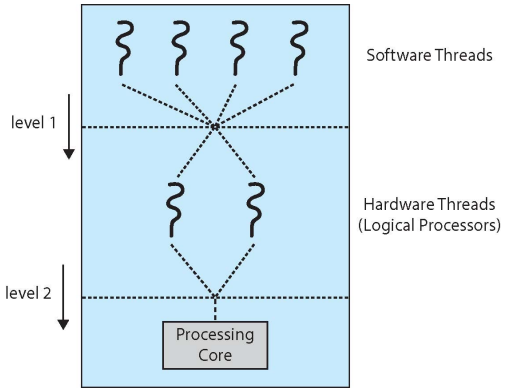
When a processor accesses memory, spends a significant amount of time waiting for data to become available - a **memory stall.**

Stalls occur because modern processors operate at **much faster speeds** than memory.



A **processor** can spend up to **50%** of its time **waiting** for data to become available from memory.

Chip multithreading/hyperthreading



Hyper-threading works by allowing one CPU core to run multiple threads at once, as opposed to one thread per core.

Ten-core Intel® Core™ i9-10900K processor has 20 threads when Hyper-Threading is enabled.

To mitigate this, hardware designs have 2 or more **hardware threads** allocated to **each core**.

- If one hardware thread stalls while waiting for memory, the core can switch to another thread.
- Each thread appears as a **logical CPU** that is available to run a software thread.

Technique known as **chip multithreading** (CMT)

Some processors support simultaneous multithreading (SMT), such as **Intel Hyper-Threading**.

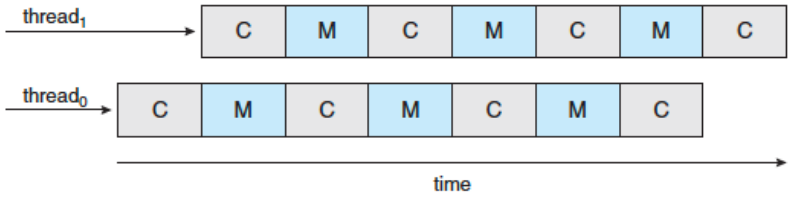
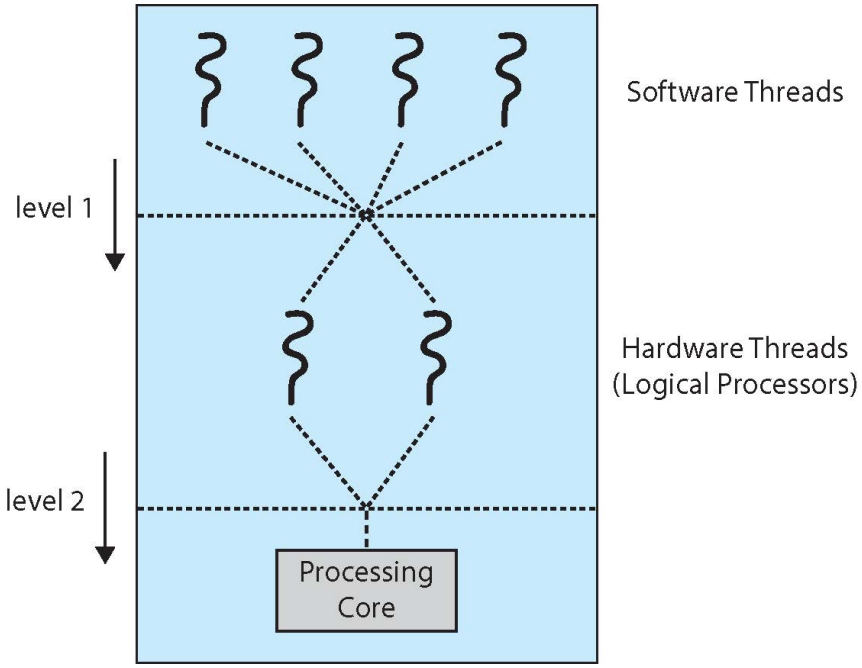


Figure 5.13 Multithreaded multicore system.

Apple M-series chips do not expose SMT in the same way as many Intel CPUs.

https://www.intel.com/content/www/us/en/gaming/resources/hyper-threading.html

Now two levels of scheduling



Level 1: OS chooses which software thread to run on each hardware thread.

- Can use any of the algorithms discussed.

Level 2: thread selection at the hardware level

- Can be simple RR
- Or **priority based** (e.g. hardware thread has a dynamic urgency value ranging from 0 to 7, with 0 representing the lowest urgency and 7 the highest.)

SMP Load balancing

Need to ensure work is **evenly distributed across processors.**

only an issue with per-core run queues.

Approaches:

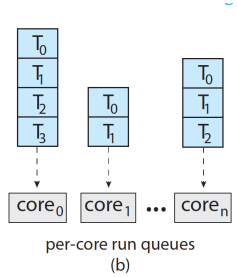
Push migration:

- Supervisor task periodically checks load on each processor and— moves (pushes) threads from overloaded to idle or less-busy processors.

Pull migration:

- an idle processor pulls a waiting task from a busy processor

Often implemented together, e.g. in the Linux CFS scheduler



per-core run queues
(b)

Push migration:
CPU 0 (busy) ---- task ----> CPU 1 (idle)
[A][B][C] [ ]

Pull migration:
CPU 0 (idle) <---- task ---- CPU 1 (busy)
[ ] [A][B][C]

SMP Processor Affinity

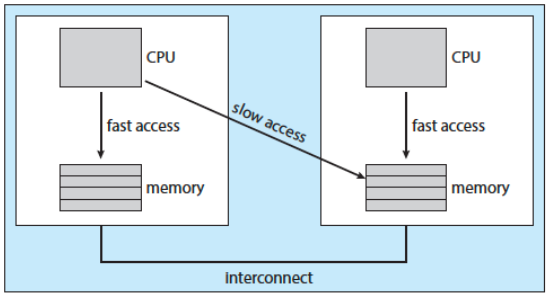


Figure 5.16 NUMA and CPU scheduling.

Avoid migrating a thread from one processor to another;

try to keep a thread running on the same processor to take advantage of a **warm cache**.

Tradeoffs between **processor affinity** and **load balancing**.

Worse with architecture featuring *non-uniform memory access* (**NUMA**) where there are two physical processor chips each with their own CPU and local memory.

- On NUMA machines, memory is physically distributed, so running a thread near its memory can be faster than moving it to another socket.
- very complex scheduling algorithms

CFS: Load balancing

Load of each thread is a combination of thread's **priority** and its average rate of **CPU** utilization.

- a thread with high priority that is I/O-bound with little CPU usage, will have a **low load**,
- similar to low-priority thread with high CPU utilization.

Load of a queue is sum of loads of all threads in the queue

- **balancing** ensures that all queues have approximately the same load.

CFS load-balancing is **NUMA-aware**…what does this mean?

CFS: Load balancing

CFS is also NUMAaware and minimizes the migration of threads by defining **scheduling domains** and **NUMA nodes.**

CFS balances loads within domains

Migrating threads between separate NUMA nodes would only occur under severe load imbalances.

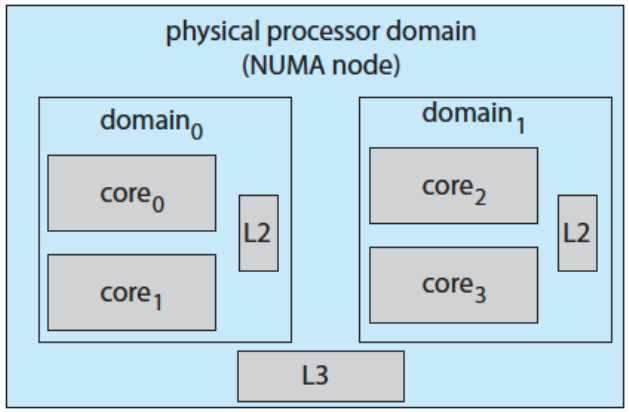


Figure 5.27 NUMA-aware load balancing with Linux CFS scheduler.

Which scheduling algorithm is best for *human* job scheduling?

- FCFS, SJF, RR, Priority, Multi-level queue?
- Pre-emptive or non-preemptive?

Criteria?

CPU utilization – keep the CPU as busy as possible

Throughput – number of processes per time unit

Turnaround time – (= interval from the time of **submission** of a process to the time of **completion**) i.e. total amount of time to execute a particular process

Waiting time – total amount of time a process has been waiting in the ready queue

Response time – (= **time to start responding**) amount of time it takes from when a request was submitted until the **first response** is produced, not final output (for time-sharing environment)

Other **desirable criteria**

- Predictability
  - **the variance** in a metric like response time
- **Fairness**
- **Lack of starvation**